

UTN FRBA – SSL – Examen Final – 2026-05-19

Apellido, Nombre:		Legajo:		Nota:	
-------------------	--	---------	--	-------	--



- Resuelva el examen en tinta y en esta hoja; no se aceptan documentos adicionales.
- Durante el examen no se responden consultas; si lo necesita, escriba hipótesis de trabajo, las cuales también se evalúan.

- Indique si las siguientes afirmaciones son verdaderas o falsas. Justifique su respuesta:
 - (1 punto) Una gramática ambigua puede generar más de un árbol de derivación para la misma cadena.
 - (1 punto) La tabla de símbolos se crea y utiliza exclusivamente durante la etapa de análisis léxico.
 - (1 punto) En C, el alcance de las variables se determina en tiempo de compilación.
 - (1 punto) Una ER es capaz de reconocer lenguajes que requieren pares balanceados como $a^n b^n$.
- Dada la declaración C: `char *(*f)(int *)`;
 - (1 punto) Indique la semántica de `f`.
 - (1 punto) Indique si `f` es *valor-l*, *valor-l modificable*, o ninguno de los anteriores. Justifique.
- Dada la siguiente gramática, con `S` como axioma:
`S → OS`
`S → A1`
`A → ε`
 - (1 punto) Indique el tipo de lenguaje que genera y el tipo de gramática.
 - (1 punto) Escriba una ERX correspondiente.
- Dado el siguiente fragmento en C:
`char a[] = {64, 65, 66}, *p = a;`
 - (1 punto) ¿Qué valor genera la expresión `*(p+2)`?
 - (1 punto) Explique la diferencia conceptual entre las expresiones `sizeof a` y `sizeof p`.
- (Punto Extra) Dada la siguiente invocación en la línea de comandos:
`./test --mode debug file.txt`
Justifique el valor de `argc[argv]`

1. Una Resolución

1.
 - a. Verdadero. Es la definición formal de ambigüedad en gramáticas independientes del contexto. Si existe al menos una cadena en el lenguaje que posea más de un árbol de derivación distinto (o lo que es lo mismo, más de una derivación por la izquierda o más de una derivación por la derecha), la gramática es ambigua.
 - b. Falso. La tabla de símbolos se crea durante el análisis léxico, y se usa durante el resto del proceso de compilación. Se consulta y enriquece durante el análisis sintáctico y el análisis semántico (para validación de tipos, control de declaraciones previas, verificación de alcances), e incluso es fundamental para la etapa de generación de código.
 - c. Verdadero. El lenguaje C utiliza un sistema de alcance (scope) estático, también llamado léxico. El entorno y la visibilidad de una variable se resuelven en tiempo de compilación observando la estructura de los bloques de código fuente, sin importar cómo sea la secuencia de llamadas de funciones en tiempo de ejecución.
 - d. Falso. Las expresiones regulares (y los autómatas finitos que las implementan) solo pueden reconocer lenguajes regulares (Tipo 3). El lenguaje del tipo $a^n b^n$ requiere aparear pares balanceados, lo que implica "contar" cuántas letras a vinieron para asegurar la misma cantidad de letras b . Para esto se necesita "memoria" adicional, como la que provee un autómata de pila, ubicando al lenguaje en la categoría de libre de contexto (Tipo 2).
2.
 - a. `f`: Puntero a una función, que retorna puntero a `char` y tiene como parámetro un puntero a `int`.
 - b. *Valor-l modifiable*: `f` es una expresión que localiza un objeto de tipo puntero, el valor de ese objeto es modificable.
3.
 - a. La GF genera un LR (tipo 3), y es tipo GIC (tipo 2). Por definición, para que una gramática sea regular, *todas* sus producciones deben ser estrictamente lineales a la derecha o *todas* estrictamente lineales a la izquierda.
 - b. Una ERX es: 0^*1 .
4.
 - a. 66 ó 'B'
 - b. Tamaño de una variable arreglo versus tamaño de una variable puntero.
5. `argc[argv] == nullptr`.
La subindicación es conmutativa, queda como ejercicio porqué. `argv` apunta a un arreglo de punteros a puntero de `char`:

```
char *a[] = {". /test" , "--mode", "debug", "file.txt", nullptr};
auto argv = a;
auto argc = sizeof a / sizeof *a - 1;
assert(argc[argv] == nullptr);
```

v1.0.0-rc.1+2026-05-24